CSL 100/ MAL 100

Tutorial 10

Structure and Union

Write a program to define a Student structure with members: name, roll_no, and marks. Create an array of Student structures and allow the user to input data for each student. Then, display the data in a tabular format

Example

```
Enter the number of students: 3
Enter details for student 1:
Name: Alice
Roll Number: 101
Marks: 89.5
Enter details for student 2:
Name: Bob
Roll Number: 102
Marks: 75.25
Enter details for student 3:
Name: Charlie
Roll Number: 103
Marks: 92.0
Name                Roll No    Marks
-------------------------------------------------
Alice               101        89.50
Bob                 102        75.25
Charlie             103        92.00
```

```c
#include <stdio.h>

// Define the Student structure
struct Student {
    char name[50];
    int roll_no;
    float marks;
};
int main() {
    int n;
    printf("Enter the number of students: ");
    scanf("%d", &n);
    struct Student students[n];
    for (int i = 0; i < n; i++) {
        printf("\nEnter details for student %d:\n", i + 1);
        printf("Name: ");
        scanf(" %[^\n]s", students[i].name);
        printf("Roll Number: ");
        scanf("%d", &students[i].roll_no);
        printf("Marks: ");
        scanf("%f", &students[i].marks);
    }
    // Display student data in tabular format
    printf("\n%-20s %-10s %-10s\n", "Name", "Roll No",
"Marks");
    printf("-----------------------------------------------\n");
    for (int i = 0; i < n; i++) {
        printf("%-20s %-10d %-10.2f\n", students[i].name,
students[i].roll_no, students[i].marks);
    }

    return 0;
}
```

Define an Employee structure that includes name, id, and a nested structure Address (with city, state, zipcode). Create a function to display an employee's details, including their address.

Example
Enter employee name: John Doe
Enter employee ID: 12345
Enter city: New York
Enter state: NY
Enter zipcode: 10001

Employee Details:
Name: John Doe
ID: 12345
Address:
  City: New York
  State: NY
  Zipcode: 10001

```c
#include <stdio.h>

// Define the Address structure
struct Address {
    char city[50];
    char state[50];
    int zipcode;
};
Address structure
struct Employee {
    char name[50];
    int id;
    struct Address address;
};
void displayEmployee(struct Employee emp) {
    printf("Employee Details:\n");
    printf("Name: %s\n", emp.name);
    printf("ID: %d\n", emp.id);
    printf("Address:\n");
    printf("  City: %s\n", emp.address.city);
    printf("  State: %s\n", emp.address.state);
    printf("  Zipcode: %d\n", emp.address.zipcode);}
```

```c
int main() {
    struct Employee emp;

    // Input employee details
    printf("Enter employee name: ");
    scanf(" %[^\n]s", emp.name);
    printf("Enter employee ID: ");
    scanf("%d", &emp.id);

    // Input address details
    printf("Enter city: ");
    scanf(" %[^\n]s", emp.address.city);
    printf("Enter state: ");
    scanf(" %[^\n]s", emp.address.state);
    printf("Enter zipcode: ");
    scanf("%d", &emp.address.zipcode);

    // Display employee details
    displayEmployee(emp);

    return 0;
}
```

Define a Complex structure with two members: real and imag (both float). Write functions to add two complex numbers. Display the results.

Example
Enter the real and imaginary parts of the first complex number: 45 20
Enter the real and imaginary parts of the second complex number: 20 1

Addition: 65.00 + 21.00i

```c
#include <stdio.h>

// Define the Complex structure
struct Complex {
    float real;
    float imag;
};

struct Complex add(struct Complex a, struct Complex b) {
    struct Complex result;
    result.real = a.real + b.real;
    result.imag = a.imag + b.imag;
    return result;
}

void display(struct Complex c) {
    if (c.imag >= 0)
        printf("%.2f + %.2fi\n", c.real, c.imag);
    else
        printf("%.2f - %.2fi\n", c.real, -c.imag);
}

int main() {
    struct Complex num1, num2, result;

    // Input two complex numbers
    printf("Enter the real and imaginary parts of the first complex number: ");
    scanf("%f %f", &num1.real, &num1.imag);
    printf("Enter the real and imaginary parts of the second complex number: ");
    scanf("%f %f", &num2.real, &num2.imag);

    // Perform addition
    result = add(num1, num2);
    printf("\nResult of Addition: ");
    display(result);

    return 0;
}
```

Create a union that can store an int, float, and char. Write a program to assign values of each type to the union and display them. Notice how only one value can be stored at a time

Example
After assigning integer:
Int: 42
Float: 0.00
Char:

After assigning float:
Int: 1078523331
Float: 3.14
Char:

After assigning char:
Int: 65
Float: 0.00
Char: A

```c
#include <stdio.h>

// Define the union with int, float, and char members
union Data {
    int i;
    float f;
    char c;
};
int main() {
    union Data data;
    data.i = 42;
    printf("After assigning integer:\n");
    printf("Int: %d\n", data.i);
    printf("Float: %.2f\n", data.f);
    printf("Char: %c\n", data.c);

    data.f = 3.14;
    printf("\nAfter assigning float:\n");
    printf("Int: %d\n", data.i);
    printf("Float: %.2f\n", data.f);
    printf("Char: %c\n", data.c);

    data.c = 'A';
    printf("\nAfter assigning char:\n");
    printf("Int: %d\n", data.i);
    printf("Float: %.2f\n", data.f);
    printf("Char: %c\n", data.c);

    return 0;
}
```

Difference Between Structure and union

```
#include <stdio.h>
#include <string.h>
struct ExampleStruct {
    int integer;
    float decimal;
    char character;
};
union ExampleUnion {
    int integer;
    float decimal;
    char character;
};

int main() {
    struct ExampleStruct s;
    union ExampleUnion u;
    printf("Size of structure: %lu bytes\n", sizeof(s));
    printf("Size of union: %lu bytes\n", sizeof(u));
    s.integer = 10;
    s.decimal = 3.14;
    s.character = 'A';
    printf("\nStructure members:\n");
    printf("Integer: %d\n", s.integer);
    printf("Decimal: %.2f\n", s.decimal);
    printf("Character: %c\n", s.character);
    u.integer = 10;
    printf("\nUnion members after setting integer:\n");
    printf("Integer: %d\n", u.integer);

    u.decimal = 3.14;
    printf("Union members after setting decimal:\n");
    printf("Decimal: %.2f\n", u.decimal);
    printf("Integer (overwritten): %d\n", u.integer);
    u.character = 'A';
    printf("Union members after setting character:\n");
    printf("Character: %c\n", u.character);
    printf("Decimal (overwritten): %.2f\n", u.decimal);

    return 0;
}
```

QUIZ

Write a program in C to manage student records. The program should: Define a Student structure containing the following fields: name (a string to store the student's name), roll (an integer to store the roll number), marks (a float to store the student's marks).Take input for 10 students, including each student's name, roll number, and marks. Sort the array of students in descending order of marks (from highest to lowest). Display the sorted list of students, showing their name, roll number, and marks

Enter details for 10 students:
Enter details for student 1:
Name: Alice
Roll Number: 101
Marks: 85.5
Enter details for student 2:
Name: Bob
Roll Number: 102
Marks: 90.0
Enter details for student 3:
Name: Charlie
Roll Number: 103
Marks: 75.5
…

Students sorted by marks (highest to lowest):

Name	Roll No	Marks
George	107	95.00
Edward	105	91.00
Bob	102	90.00
Diana	104	88.00
Jack	110	87.50
Alice	101	85.50
Hannah	108	82.00
Ian	109	78.00
Charlie	103	75.50
Fiona	106	69.50